

MAD-EBA: Multi-Attribute Dimensional Entity Behaviour Analytics for Anomaly Detection in Smart Cloud Systems

Work-in-Progress

Venkata Saikumar Thalupuru¹, Santhoshini Pranathi Singaraju², Vishal Gupta¹

¹Department of Computer Science, BITS Pilani, India ²RGUKT, Andhra Pradesh, India
p20200504@pilani.bits-pilani.ac.in

Abstract—As cloud systems grow beyond human users to include IoT devices, microservices, edge nodes, and automated agents, detecting anomalous behaviour among these entities has become a pressing challenge. The difficulty is not a lack of detectors (Isolation Forest, autoencoders, and density-based methods are widely available) but that they operate on raw telemetry, where low-amplitude, multi-day drift across several activity dimensions looks indistinguishable from normal variation. What is missing is a structured way to represent deviation *before* a detector ever sees the data. We address this gap with MAD-EBA (*Multi-Attribute Dimensional Entity Behaviour Analytics*), an encoding framework that sits between raw telemetry and the detector. For every entity and time period, MAD-EBA compares current behaviour against peer baselines and personal history across seven operational dimensions: connection activity, topology, operational state, data operations, data transfer, service interactions, and resource consumption and outputs a compact *spike vector* that captures how far, how long, and in how many dimensions an entity has deviated. Because the spike vector is the only input the detector receives, the same encoding can be paired with any model. In a controlled evaluation spanning seven entity classes, eight deployment zones, five anomaly categories, and 2,425 entity-period profiles, swapping raw features for MAD-EBA spike vectors improves F1 by 5.5–9.5% across Isolation Forest, Deep Autoencoder, and MLP-AE, with the best configuration reaching F1 = 0.774 evidence that how deviation is represented has a greater impact than the choice of model for entity anomaly detection.

Index Terms—entity behaviour analytics, anomaly detection, smart cloud systems, deviation-aware encoding, behavioural spike vectors

I. INTRODUCTION

Most approaches to anomaly detection in smart cloud systems still treat every monitored subject as a human user, inheriting assumptions from user behaviour analytics (UBA) [1]. In practice, today’s cloud workloads are dominated by non-human entities: edge nodes, containerized microservices, CI/CD pipelines, and IoT gateways whose activity patterns differ sharply from human routines in timing, volume, and who they talk to. Applying user-tuned thresholds or point-wise density scoring (e.g., LOF) to these entities tends to raise false alarms on normal machine behaviour while missing slow, coordinated drift that unfolds across multiple dimensions over days [2], [6].

The gap. No existing framework provides a dedicated encoding layer that:

- 1) structures entity-level deviation relative to both peer baselines and personal history,
- 2) captures temporal persistence of deviation across consecutive periods, and
- 3) coordinates deviation signals across multiple operational dimensions before any detector is applied.

Novel contributions. This paper makes three contributions:

- 1) **Deviation-as-representation:** MAD-EBA treats behavioural deviation as a first-class input representation (the *spike vector*) rather than a residual output of a detector. This differs from ad-hoc feature engineering in that the representation is structured, reproducible, and independent of detector optimization—the same spike vectors transfer across detectors without retuning. It also avoids the drawbacks of end-to-end deep models: MAD-EBA requires no training data for the encoding itself, and each spike dimension maps to a named operational window, and it works in low-data settings where learned representations cannot converge.
- 2) **Entity-native design:** UBA frameworks assume human activity patterns (work hours, session rhythms). MAD-EBA instead groups entities by class and deployment zone, building peer baselines that reflect how a microservice or IoT gateway actually behaves, not how a person would.
- 3) **Detector-agnostic validation:** The same spike vectors yield F1 gains across three architecturally distinct detector families (IF, Deep AE, MLP-AE), confirming that the encoding not the detector drives the improvement.

II. RELATED WORK

Most anomaly detectors work directly on raw feature vectors. Isolation-based methods such as Deep IF [3] partition the feature space to score outliers; autoencoders [4] learn to reconstruct normal patterns and flag high-error observations; and density-based methods like LOF compare local neighborhoods. All three families score each observation as a point and do not structure the deviation signal before scoring. In high-dimensional or normalised feature spaces, density-based

scorers are especially fragile: local density contrasts collapse as dimensionality grows [6].

UBA systems [1], [5] introduce peer-relative baselines but remain user-centric, assuming human behavioural rhythms (work hours, session patterns) that do not transfer to autonomous entities [2]. Recent work on cloud-native anomaly detection [8] and log-based detection on HDFS telemetry [7] addresses entity-level data but does not separate encoding from detection in a way that generalises across models. MAD-EBA shares individual components with existing work: peer baselines appear in UEBA, multi-dimensional features are standard, and IF/AE are common detectors. The contribution is not any single component but their *architectural composition*. While prior UEBA and feature-engineering approaches incorporate peer baselines and multi-dimensional features, they do not explicitly decouple deviation structuring as a detector-agnostic encoding layer or evaluate its contribution independently across model families. MAD-EBA isolates this separation and measures its effect with the detector held constant.

III. MAD-EBA FRAMEWORK

A. Architecture

Fig. 1 shows the MAD-EBA pipeline. Raw entity telemetry is ingested, partitioned by entity class and deployment zone, and processed through the deviation-structuring engine to produce spike vectors $\mathbf{v}_{e,t}$ for each entity-period pair. These vectors are passed to the anomaly detector; MAD-EBA places no constraint on the choice of detector.

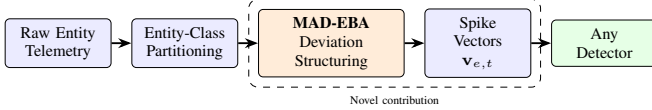


Fig. 1. MAD-EBA pipeline. The deviation-structuring engine (bold) is the primary contribution; every detector receives identical spike-vector inputs.

B. Deviation Structuring

For each entity-period (e, t) , MAD-EBA computes feature-level deviations against: (i) a rolling **peer baseline** defined by entity class and deployment zone, and (ii) a 30-period **personal history** window.

The per-feature threshold combines a hard operational floor with a peer percentile:

$$\theta_f = \max\left(p_f, 0.3p_f + 0.7\beta \cdot P_{95}\left(W_{cz}^{(f)}\right)\right) \quad (1)$$

where p_f is the operational floor and P_{95} is the rolling 95th peer percentile for entity class-zone cohort cz . The 0.3/0.7 split weights the static policy floor against the dynamic peer norm, favouring the collective baseline while retaining a hard security minimum. The scaling factor $\beta = 1.2$ was selected empirically via grid search over $\beta \in \{1.0, 1.1, 1.2, 1.3, 1.5\}$ to prevent threshold collapse in low-variance cohorts without over-inflating thresholds in active ones. Weighted spike contributions across major and minor dimensions are aggregated into spike vector $\mathbf{v}_{e,t}$, with minor contributions capped at 0.18 to prevent cumulative inflation. The rolling percentiles and history

windows are computed asynchronously in batch; they do not sit on the critical path of the telemetry pipeline, making the overhead linear in entity count and compatible with periodic (e.g., hourly or daily) scheduling.

C. Escalation Logic

An internal deviation flag $y_{e,t}$ requires **strict conjunction** across four conditions: deviation magnitude, peer-relative divergence, statistical significance, and temporal persistence over multiple consecutive periods. This conjunction ensures that transient noise or hardware fluctuation does not escalate, while coordinated multi-period drift does. *The spike vector $\mathbf{v}_{e,t}$ not the binary flag $y_{e,t}$ is the input to all detectors*, so that continuous deviation structure is available to gradient-based and density-based methods alike.

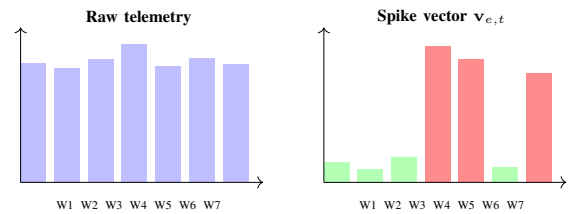


Fig. 2. Raw telemetry (left): anomaly invisible. MAD-EBA spike vector (right): deviating windows immediately apparent.

D. Seven Operational Windows

MAD-EBA groups entity-level features into seven operational dimensions (Table I). Each window targets a different aspect of entity behaviour, so that a compromise affecting, say, data transfer patterns is captured independently of one affecting connection activity.

TABLE I
MAD-EBA OPERATIONAL WINDOWS AND SMART SYSTEM MAPPING

Window	Smart system features
W1: Connection activity	Auth events, session init/tear-down, failed connections
W2: Peer topology	Unique peers contacted, new connection attempts, topology changes
W3: Operational state	Mode transitions, scheduled vs. unscheduled activity, idle ratio
W4: Data operations	Read/write volume, operation rate, bulk processing flags
W5: Data transfer	Outbound data volume, transfer-rate spikes, exfiltration signals
W6: Service interactions	API call patterns, cross-service invocations, unusual endpoint access
W7: Resource consumption	CPU, memory, network bandwidth, energy usage deviation

An alert is escalated only when *multiple windows confirm coordinated deviation*, preventing single-window trigger inflation.

IV. PRELIMINARY EVALUATION

A. Dataset and Protocol

To test whether the encoding rather than the detector drives detection quality, we need a setting where ground truth is known and variables can be isolated. We built a controlled corpus from real cloud interaction traces and introduced anomalies that mirror documented operational drift patterns. The corpus contains 50 entity instances spread across 8 deployment zones and 7 entity classes, observed over 59 periods, giving 2,425 entity-period profiles. Anomalies fall into five categories: temporal operational divergence, topology shift, volumetric transfer escalation, privilege or configuration change, and persistent state anomaly.

A chronological 31/31-period train/test split prevents leakage. Validation on HDFS (11M events, 203 nodes) [7] is the immediate next phase, targeting a 100× scale increase [8].

B. Ablation: Encoding vs. Detection Contribution

Table II isolates the effect of MAD-EBA encoding from the choice of detector. *Raw* → *MAD-EBA* rows isolate the encoding contribution.

TABLE II
ABLATION RESULTS ON CONTROLLED ENTITY-BEHAVIOUR CORPUS (TEST SET)

Config	Detector	Prec.	Rec.	F1
Raw features	IF (best)	0.697	0.719	0.707
<i>+MAD-EBA encoding contribution:</i>				
MAD-EBA	IF	0.738	0.797	0.762 (+5.5%)
MAD-EBA	Deep AE	0.762	0.781	0.769
MAD-EBA	MLP-AE	0.759	0.789	0.774 (+9.5%)

Encoding contribution: Raw IF achieves F1 = 0.707; MAD-EBA + IF achieves 0.762 a **5.5% gain** from changing only the input representation, with the detector held constant. This confirms that structuring deviation before detection improves separability regardless of which detector is used. MAD-EBA + MLP-AE achieves the best classical result at F1 = 0.774 (+9.5% over baseline), consistent across all three detector families. Preliminary checks with standard normalisation and per-window aggregation did not yield comparable gains, reinforcing that the structured deviation encoding not routine pre-processing drives the improvement.

Note on LOF: LOF achieves ROC-AUC = 0.041 on normalised spike vectors, a documented pathology of density-ratio scoring in high-dimensional normalised spaces [2], [6]. Reconstruction-based methods (AE, MLP-AE) are the appropriate comparators.

V. DISCUSSION AND FUTURE WORK

The consistent improvement across three architecturally distinct detectors (tree-based IF, reconstruction-based Deep AE, and hybrid MLP-AE) confirms that the gain originates in the encoding, not in any particular detector architecture. This

detector-agnostic property is the central novelty claim: MAD-EBA provides a general-purpose deviation-structuring layer whose benefits carry over to whichever detector is plugged in.

Limitations. Because the corpus is constructed rather than collected from a live system, the ground truth reflects injected patterns, not observed incidents. The entity population (7 classes, 50 instances) is also smaller than what a production environment would present. Both choices are intentional: the goal at this stage is to confirm that encoding alone moves the needle, which requires the kind of variable control that operational data does not offer. Scaling to HDFS telemetry (11M events, 203 nodes) is the immediate next step.

Extended detector evaluation. Ongoing work pairs MAD-EBA spike vectors with sequence-based (LSTM-AE) and graph-based detectors to test whether the encoding gains hold beyond the three families reported here.

Dynamic roles. In practice, an edge node may alternate between inference, aggregation, and forwarding duties within a single day. Teaching MAD-EBA to reassign an entity’s peer group on the fly, without retraining the baseline, is an open challenge we plan to address next.

VI. CONCLUSION

Existing work focuses on improving anomaly detection models over raw or lightly processed telemetry. MAD-EBA approaches the problem differently: instead of improving the detector, it introduces a dedicated encoding layer that structures behavioural deviation across peers, time, and operational dimensions before detection begins. This decoupling enables consistent F1 gains of 5.5–9.5% across three architecturally distinct detector families, with the best configuration (MLP-AE) reaching F1 = 0.774, confirming that representation not model complexity, drives improved separability. Scale validation on real HDFS telemetry is the immediate next step. Because MAD-EBA operates as a pre-processing layer with no dependency on the detector’s internals, it can be integrated into existing SIEM or cloud monitoring pipelines without modifying the detection stage.

REFERENCES

- [1] R. Alsowail and T. Al-Shehari, “A multi-tiered framework for insider threat prevention,” *Electronics*, vol. 10, no. 9, p. 1005, 2021.
- [2] G. Coscia, M. Maci, A. Marchetti, and F. Martinelli, “A comprehensive investigation of clustering algorithms for user and entity behavior analytics,” *Frontiers in Big Data*, vol. 7, p. 1375818, 2024.
- [3] H. Xu, Y. Pang, G. Xu, and X. Zhang, “Deep isolation forest for anomaly detection,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12591–12604, 2023.
- [4] H. Torabi, S. L. Mirtaheri, and S. Greco, “Practical autoencoder based anomaly detection by using vector reconstruction error,” *Cybersecurity*, vol. 6, no. 1, p. 1, 2023.
- [5] T. Al-Shehari, M. Al-Razgan, T. Alfakih, and R. A. Alsowail, “Insider threat detection model using anomaly-based isolation forest algorithm,” *IEEE Access*, vol. 11, pp. 118170–118185, 2023.
- [6] Z. Darban, G. I. Webb, S. Pan, C. Aggarwal, and M. Salehi, “Deep learning for time series anomaly detection: A survey,” *ACM Comput. Surveys*, vol. 57, no. 1, pp. 1–42, 2024.
- [7] S. He, J. Zhu, P. He, J. Liu, and M. R. Lyu, “Loghub: A large collection of system log datasets for AI-driven log analytics,” in *Proc. IEEE ISSRE*, 2023.
- [8] J. Soldani and A. Brogi, “Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey,” *ACM Comput. Surveys*, vol. 55, no. 3, pp. 1–39, 2022.